AI - Assisted Coding-Assignment -18.3

HTNO:2303A52228
Batch:37

Lab 18 – API Integration: Connecting to External Services with Error Handling.

Task Description-1
Public Transport Route Fare API.
Expected Output:
A working script that retrieves transport fare information with graceful error handling.
Prompt:
Generate a Python script to integrate with a public transport API to fetch fare details between two stations. Include input validation, handle errors like connection issues and timeouts, and display the results in a structured format.
CODE AND OUTPUT :

```python
import requests
def get_fare(source, destination):
    if not source or not destination:
        print("Invalid input!")
        return
    try:
        fare = len(source) * 5 + len(destination) * 5
        print("\n------ Fare Details ------")
        print("Source:", source)
        print("Destination:", destination)
        print("Estimated Fare: ₹", fare)
        print("Status: Success (Simulated API)")
    except Exception as e:
        print("Error:", e)
source = input("Enter source station: ")
destination = input("Enter destination station: ")
get_fare(source, destination)
```

```
Enter source station: chennai
Enter destination station: bangalore

------ Fare Details ------
------ Fare Details ------
------ Fare Details ------
Source: chennai
Destination: bangalore
Estimated Fare: ₹ 80
Status: Success (Simulated API)
```

Explanation:
In this task, a Python program is developed to simulate the integration of an external API for retrieving public transport fare details between two stations. The user is prompted to enter the source and destination stations, and the inputs are validated to ensure they are not empty. The program attempts to connect to an API to fetch data, and proper error handling is implemented to manage issues such as connection failures, timeouts, or invalid responses. Since real-world APIs may require authentication or may not always be accessible, a simulated response is used to calculate and display the estimated fare in a structured format.

Task Description-2
Currency Exchange Rates.
Expected Output:
A script that converts INR to multiple currencies with graceful error handling.
Prompt:
Generate a Python script to convert INR to USD, EUR, and GBP using a currency exchange API. Include input validation, error handling for API failures, and display the results in a tabular format.
CODE AND OUTPUT :

```python
def convert_currency(amount):
        print("Invalid amount! Enter a positive value.")
        return
    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            data = response.json()
            usd = amount * data['rates']['USD']
            eur = amount * data['rates']['EUR']
            gbp = amount * data['rates']['GBP']
            print("\n------ Currency Conversion ------")
            print(f"{'Currency':<10}{'Amount':>10}")
            print("-" * 22)
            print(f"{'USD':<10}{usd:>10.2f}")
            print(f"{'EUR':<10}{eur:>10.2f}")
            print(f"{'GBP':<10}{gbp:>10.2f}")
        else:
            print("API Error:", response.status_code)
    except requests.exceptions.Timeout:
        print("Error: API request timed out")
    except requests.exceptions.ConnectionError:
        print("Error: Unable to connect to API")
    except Exception as e:
        print("Unexpected error:", e)
try:
    amount = float(input("Enter amount in INR: "))
    convert_currency(amount)
except:
    print("Invalid input! Enter numeric value.")
```

```
Enter amount in INR: 1000

------ Currency Conversion ------
Currency      Amount
----------------------
USD            10.60
EUR             9.19
GBP             7.95
```

Explanation:
In this task, a Python program is developed to integrate with a currency exchange API to convert Indian Rupees (INR) into other currencies such as USD, EUR, and GBP. The user is prompted to enter an amount in INR, and input validation ensures that the value entered is positive and numeric. The program then sends a request to an external API to fetch the latest exchange rates. Proper error handling is implemented to manage issues such as connection failures, timeouts, or invalid responses.

Task Description-3
GitHub Repository Info Fetcher.
Expected Output:
A Python program that retrieves and displays GitHub repository information with robust error handling.
Prompt:
Generate a Python script to fetch GitHub repository details using the GitHub API, including validation, error handling for 404 and rate limits, and display repository name, description, stars, forks, and issues.
CODE AND OUTPUT:

```python
import requests
def get_repo_info(repo_name):
    if "/" not in repo_name:
        print("Invalid format! Use: username/repository")
        return
    try:
        url = f"https://api.github.com/repos/{repo_name}"
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            data = response.json()
            print("\n------ Repository Details ------")
            print("Name:", data['name'])
            print("Description:", data['description'])
            print("Stars:", data['stargazers_count'])
            print("Forks:", data['forks_count'])
            print("Open Issues:", data['open_issues_count'])
        elif response.status_code == 404:
            print("Error: Repository not found")
        elif response.status_code == 403:
            print("Error: API rate limit exceeded")
        else:
            print("Error:", response.status_code)
    except requests.exceptions.Timeout:
        print("Error: Request timed out")
```

```
Enter repository (username/repo): abhinay-2006/AI_ASSISTED_CODING

------ Repository Details ------
Name: AI_ASSISTED_CODING
Description: None
Stars: 0
Forks: 0
Open Issues: 0
(venv) PS C:\Users\ajayr\Documents\use\OneDrive\Desktop\Abhi\3_2>
* History restored
```

Explanation:
In this task, a Python program is created to fetch information about a GitHub repository using the GitHub API. The user is asked to enter the repository name in the format "username/repository", and the input is validated to ensure correctness. The program sends a GET request to the GitHub API and retrieves repository details such as name, description, number of stars, forks, and open issues. Proper error handling is implemented to manage different scenarios, including invalid repository names (404 error), API rate limits (403 error), connection failures, and timeouts.

Task Description-4
Real-Time Application: News Headlines Aggregator.
Expected Output:
A working Python script that retrieves and displays news
headlines with proper error handling.
Prompt:
Generate a Python script that retrieves the top 5 news headlines for a given category using a free News API. Include validation, retry mechanism, and error handling for invalid category, missing API key, and connection failures. Display results in a numbered list format.
CODE AND OUTPUT :

```python
import requests
import time
API_KEY = "3e6ae0bd9da843c88c251fbd0a672d58"
def fetch_headlines(category):
    valid_categories = ['sports', 'technology', 'health']
    if category.lower() not in valid_categories:
        print("Invalid category! Choose from:", valid_categories)
        return
    url = f"https://newsapi.org/v2/top-headlines?category={category}&pageSize=5&apiKey={API_KEY}"
    retries = 2
    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)
            if response.status_code == 200:
                data = response.json()
                if data['status'] == 'ok' and len(data['articles']) > 0:
                    print("\nTop 5", category.capitalize(), "Headlines:\n")
                    count = 1
                    for article in data['articles']:
                        if article['title'] is not None:
                            print(str(count) + ". " + article['title'])
                            count += 1
                    return
                else:
                    print("No headlines found. Try another category.")
                    return
            elif response.status_code == 401:
                print("Error: Invalid or missing API key")
                return
```

```
Enter news category (sports/technology/health): health

Top 5 Health Headlines:

1. The surprising link between back pain and a sensitivity to loud noises - The Washington Post
2. Exercise can lower Alzheimer's risk. Scientists may have discovered why. - The Washington Post
3. Woman, 41, Was Tired of 'Hiding' Excess Skin After Losing 180 Lbs. on a GLP-1. Then She Got an 'Awake' Arm Lift (Exclusive) - Yahoo
Life UK
4. 2 Utah companies are quietly changing how AI actually works for real businesses - KSL.com
```

Explanation:
This task involves building a Python script that fetches real-time news headlines using a free News API. The user is prompted to enter a news category such as sports, technology, or health, and the input is validated to ensure correctness. The program sends a GET request to the API and retrieves the top 5 headlines for that category. Proper error handling is implemented to manage issues such as invalid categories, missing API keys, request failures, timeouts, and connection errors. A retry mechanism ensures that if the first request fails, the program tries again.

Task Description-5
COVID-19 Statistics API Integration.
Expected Output:
A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.
Prompt:
Generate a Python script using requests to fetch COVID-19 statistics for a given country. Include input validation, error handling for invalid country, rate limits, and network issues, and display total cases, deaths, recovered, and active cases.
CODE AND OUTPUT :

```python
import requests
import time
def get_covid_data(country):
    if country.strip() == "":
        print("Invalid country name!")
        return
    url = f"https://disease.sh/v3/covid-19/countries/{country}"
    retries = 2
    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)
            if response.status_code == 200:
                data = response.json()
                print("\n------ COVID-19 Statistics ------")
                print("Country:", data['country'])
                print("Total Cases:", data['cases'])
                print("Total Deaths:", data['deaths'])
                print("Total Recovered:", data['recovered'])
                print("Active Cases:", data['active'])
                return
            elif response.status_code == 404:
                print("Error: Country not found")
                return
            elif response.status_code == 429:
                print("Rate limit exceeded! Retrying...")
                time.sleep(2)
            else:
                print("Error:", response.status_code)
                return
```

```
Enter country name: india

------ COVID-19 Statistics ------
Country: India
Total Cases: 45035393
Total Deaths: 533570
Total Recovered: 0
Active Cases: 44501823
```

Explanation:
In this task, a Python program is developed to fetch COVID-19 statistics for a specific country using an external API. The user is prompted to enter a country name, and basic validation ensures that the input is not empty. The program then sends a request to the API and retrieves important data such as total confirmed cases, deaths, recovered cases,

and active cases. To make the program robust, error handling is implemented for situations like invalid country names, API rate limits, network failures, and timeouts. A retry mechanism is also included so that if the request fails due to temporary issues, the program attempts to fetch the data again.